Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E Computer Science & Engineering	**Semester**	VI
Subject Code & Name	UCS2612 – Machine Learning Laboratory		
Academic Year	2025–2026 (Even)	**Batch**	2023–2027

Name: Mariya Joevita
Reg.No: 3122235001077
Class: CSE-B

Experiment 7

Bagging, Boosting, and Stacked Ensemble Models

1. Aim and Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

2. Dataset Description

Wisconsin Diagnostic Breast Cancer Dataset

- Total samples: 569
- Features: 30 numerical attributes
- Target classes: Malignant (M) and Benign (B)

Dataset Link: https://archive.ics.uci.edu

3. Preprocessing Steps

Before training the regression models, the dataset was preprocessed to improve data quality and ensure compatibility with the algorithms.

Loading the Dataset

The Breast Cancer Wisconsin (Diagnostic) dataset was loaded using the `ucimlrepo` library. This dataset is widely used for binary classification in medical diagnosis.

- Number of instances: 569
- Number of input features: 30 numerical features
- Target variable: Diagnosis (M = Malignant, B = Benign)

The features describe characteristics of cell nuclei extracted from breast mass images, such as radius, texture, area, smoothness, and concavity.

The objective is to build a model that classifies tumors as malignant or benign based on these features.

Encoding of Class Labels

The target variable contains categorical labels: `M` (Malignant) and `B` (Benign). Since machine learning algorithms require numerical input, the class labels were encoded using `LabelEncoder`.

- Benign (B) $\rightarrow$ 0
- Malignant (M) $\rightarrow$ 1

This transformation converts categorical values into binary numerical form suitable for classification models.

Class Label Visualization

To understand the distribution of classes, a count plot was generated. The visualization shows the number of benign and malignant cases in the dataset.

This helps identify whether the dataset is balanced or imbalanced, which is important for model evaluation.

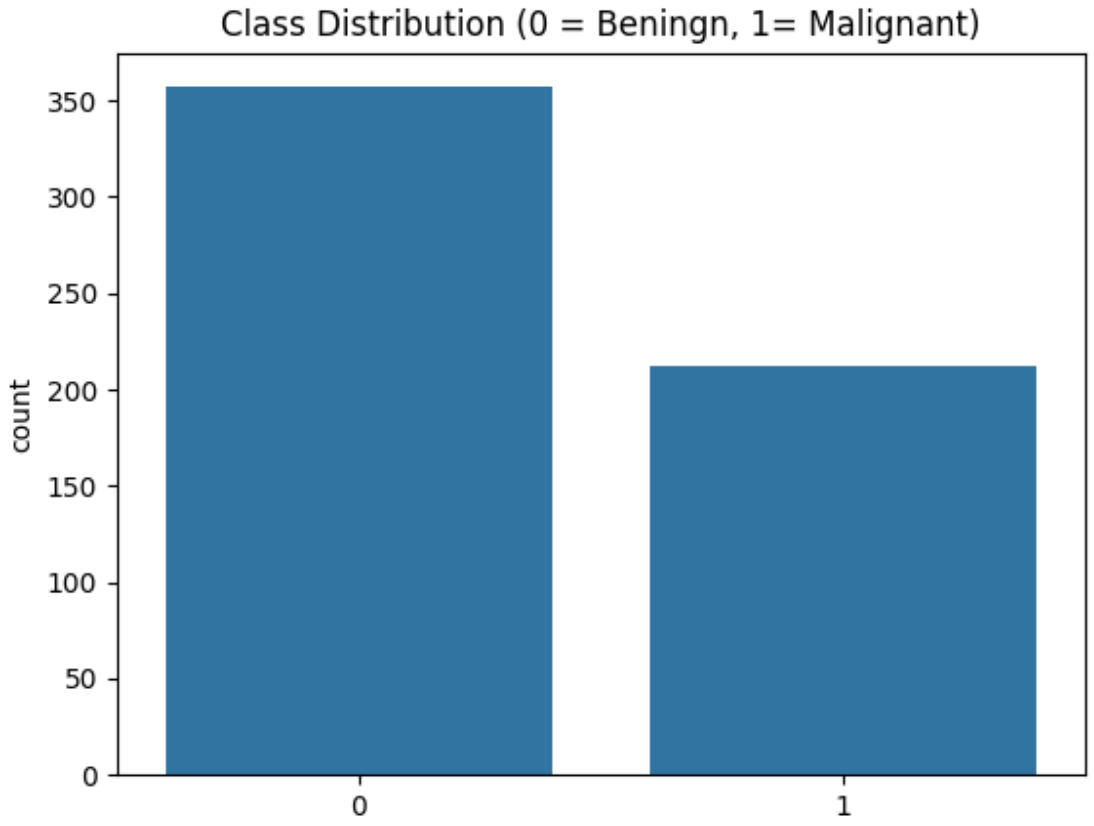


Figure 1: Distribution of Breast Cancer Type

Feature Correlation Analysis

A correlation heatmap was generated to examine relationships between numerical features.

The heatmap displays the Pearson correlation coefficients between feature pairs, helping to identify:

- Highly correlated features
- Redundant variables
- Strong relationships that may influence model performance

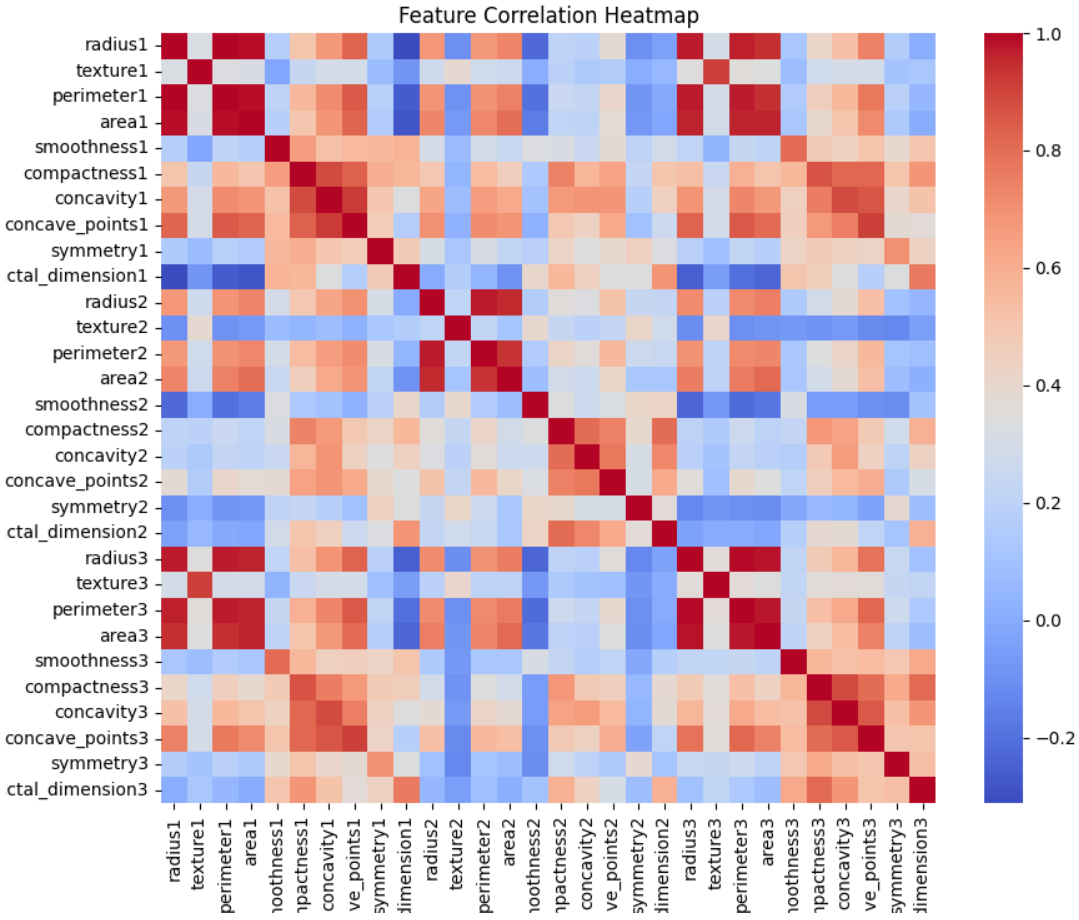


Figure 2: Correlation Heatmap of Breast Cancer Dataset Features

Understanding feature correlation assists in feature selection and improving model efficiency.

Train–Test Split

The dataset was split into training and testing sets to evaluate model performance.

- Training set: 80% of the data
- Testing set: 20% of the data

This split ensures that model evaluation is performed on unseen data.

Result of Preprocessing

After completing the preprocessing steps, the dataset was clean, numerically encoded, and properly scaled, making it suitable for training classification models.

3. Implementation Details

Feature scaling was performed using `StandardScaler` to normalize the data.

Ensemble Models Implementation

Bagging Classifier Implementation

A Bagging classifier was implemented using `BaggingClassifier` with `DecisionTreeClassifier` as the base estimator. The model was trained on the scaled training dataset.

Hyperparameter Tuning: To optimize the Bagging model, `GridSearchCV` with 5-fold cross-validation was performed. The following parameters were tuned:

- **n_estimators**: [10, 50, 100]
- **max_samples**: [0.8, 0.9, 1.0]
- **max_features**: [0.8, 1.0]

The best model was selected based on the F1-score. The optimal parameters obtained were:

```
{'max_features': 0.8, 'max_samples': 0.9, 'n_estimators': 100}
```

AdaBoost Classifier Implementation

An AdaBoost classifier was implemented using `AdaBoostClassifier` and trained on the scaled dataset.

Hyperparameter Tuning: `GridSearchCV` with 5-fold cross-validation was used. The tuned parameters include:

- **n_estimators**: [50, 100, 150]
- **learning_rate**: [0.01, 0.05, 0.1]

The best parameters based on F1-score were:

```
{'learning_rate': 0.1, 'n_estimators': 100}
```

Gradient Boosting Classifier Implementation

A Gradient Boosting classifier was implemented using `GradientBoostingClassifier`.

Hyperparameter Tuning: `GridSearchCV` with 5-fold cross-validation was applied with the following parameters:

- **n_estimators**: [50, 100]
- **learning_rate**: [0.01, 0.1]
- **max_depth**: [3, 5]

The best parameters found were:

```
{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}
```

Stacking Classifier Implementation

A stacked ensemble was constructed using `StackingClassifier`. The base learners included:

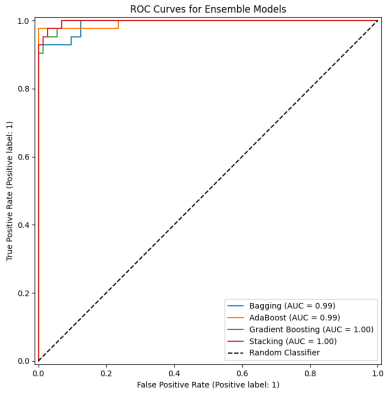
- Support Vector Classifier (`SVC`) with `probability=True`
- `GaussianNB`
- `DecisionTreeClassifier`

A `LogisticRegression` model was used as the final estimator (meta-learner) with `cv=5` for internal cross-validation. No explicit hyperparameter tuning was performed for the stacking model in this iteration.

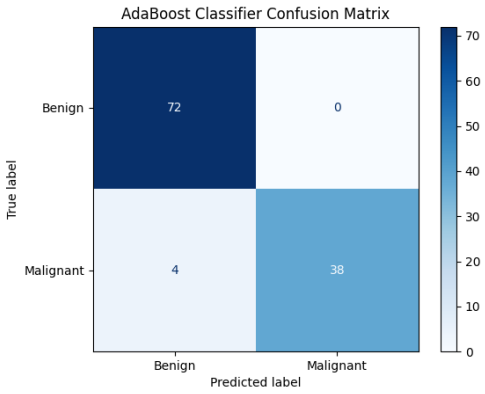
Model Evaluation and Comparison

All models (tuned Bagging, AdaBoost, Gradient Boosting, and Stacking) were evaluated on the unseen test dataset using the following metrics:

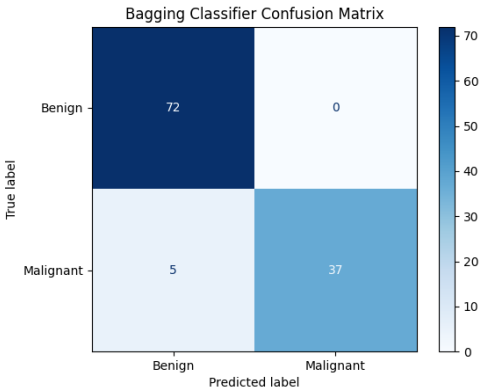
- **Accuracy**: Measures overall correctness of predictions.
- **Precision**: Measures the proportion of correctly predicted positive observations.
- **Recall**: Measures the proportion of actual positives correctly identified.
- **F1-score**: Harmonic mean of precision and recall.
- **AUC**: Area Under the ROC Curve, indicating classification performance.
- **Confusion Matrix**: Provides a summary of prediction results.



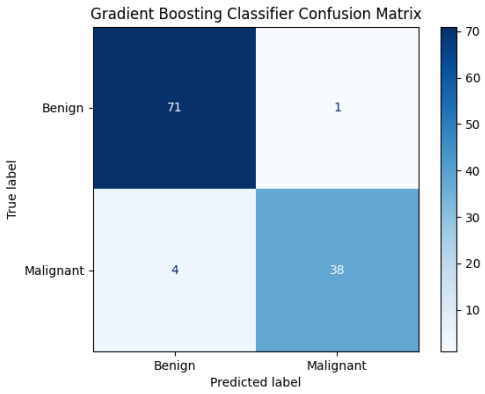
(a) ROC Plot



(b) AdaBoost



(c) Bagging



(d) Gradient Boosting

Figure 3: ROC Curve and Confusion Matrices of Ensemble Models

The results were compiled into a Pandas DataFrame for easy comparison, offering a clear overview of each model's performance across all evaluation metrics.

Ensemble Methods: Bias-Variance and Performance

1. Bagging (BaggingClassifier with Decision Tree)

Bagging reduces **variance** by training multiple decision trees on bootstrap samples and averaging their predictions. Decision trees are high-variance models, and bagging improves their stability and generalization.

Observed Performance: High accuracy and perfect precision were achieved, but recall was slightly lower, indicating some missed positive cases.

2. Boosting (AdaBoost and Gradient Boosting)

Boosting reduces **bias** by sequentially training models, where each model focuses on correcting previous errors.

AdaBoost: Achieved excellent accuracy and F1-score with perfect precision and high recall, indicating strong classification performance.

Gradient Boosting: Also performed well across all metrics, effectively capturing complex patterns and reducing bias.

3. Stacking (StackingClassifier)

Stacking combines multiple models using a meta-learner to reduce both **bias and variance**. It leverages diverse base models to improve overall performance.

Observed Performance: Achieved the highest accuracy and F1-score (along with AdaBoost), with perfect precision and high recall, showing strong generalization.

Hyperparameter Evaluation Results

Bagging Results

Table 1: Bagging Hyperparameter Evaluation

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
100	0.9	96.70	0.9564
10	0.9	96.48	0.9533
100	1.0	96.48	0.9533

Boosting Results

Table 2: Boosting Hyperparameter Evaluation

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
100	0.10	95.16	0.9356
150	0.10	94.95	0.9324
150	0.05	94.95	0.9318

Stacked Ensemble Results

Table 3: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM, NB, DT	LogisticRegression	96.49	0.9500

Performance Comparison

Table 4: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	95.61	1.0000	0.8809	0.9367
AdaBoost	96.49	1.0000	0.9047	0.9500
Gradient Boosting	95.61	0.9743	0.9047	0.9382
Stacked Ensemble	96.49	1.0000	0.9047	0.9500

Conclusion

Bagging, Boosting, and Stacked Ensemble models were implemented and evaluated. The experiment demonstrates how different ensemble strategies improve predictive performance and generalization compared to individual models.

References

- Scikit-learn: Ensemble Methods
- Bagging Classifier
- UCI Dataset